МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ВТ

отчет
по лабораторной работе № 2
по дисциплине «Распределённые системы»
Тема: Разработка и исследование микросервисной архитектуры распределенного приложения

Студенты гр. 2307 _____ Почекунин А. П.
_____ Буянов. М.Г.
_____ Подберёзский А.Д.
Преподаватель _____ Соколова Е.А.

Санкт-Петербург
2026

Цель работы: Изучение принципов микросервисной архитектуры распределенных систем, а также формирование практических навыков проектирования слабо связанных сервисов, взаимодействующих между собой по сети и использующих независимые хранилища данных.

Задание:

Спроектировать распределенное приложение на основе микросервисной архитектуры;

● Разработать не менее двух независимых микросервисов;

● Обеспечить для каждого микросервиса отдельную базу данных;

● Реализовать REST-интерфейсы для доступа к функциональности микросервисов;

● Реализовать сетевое взаимодействие между микросервисами;

● Продемонстрировать базовый сценарий совместной работы сервисов.

Предметная область выбирается студентом самостоятельно (например: управление пользователями и заказами, библиотечная система, бронирование ресурсов, учет заявок и т.п.). Распределенное приложение должно состоять как минимум из двух микросервисов, каждый из которых:

● реализует ограниченный набор бизнес-функций;

● развертывается как отдельное приложение;

● использует собственную базу данных;

● взаимодействует с другими микросервисами посредством сетевых запросов.

Требования к архитектуре системы. Проектируемая система должна соответствовать следующим требованиям:

● Использовать микросервисную архитектуру;

● Микросервисы должны быть логически и технологически независимы;

● Взаимодействие между микросервисами должно осуществляться

через сеть (HTTP, REST);

● Каждая база данных должна обслуживаться только своим микросервисом.

Функциональные требования. Минимальный набор функций системы должен включать:

● операции создания и получения данных в каждом микросервисе;

● сетевое обращение одного микросервиса к другому;

● формирование итогового ответа, использующего данные нескольких сервисов.

Требования к реализации. При выполнении лабораторной работы допускается использование любых современных технологий и средств разработки. Рекомендуется применять REST-архитектуру, формат обмена данными JSON, реляционные и нереляционные базы данных. Язык программирования, фреймворки и СУБД выбираются студентом самостоятельно.

Ход выполнения работы:

В ходе выполнения лабораторной работы я разработал распределенное приложение на основе микросервисной архитектуры для управления пользователями и их заказами. В качестве стека был использован Python+Flask, вместо баз данных был использован in-memory json, его значительно проще включить в приложение.

ИС состоит из 3 независимых компонентов: User Service на порту 5000 отвечает за операции с пользователями, Order Service на порту 5001 управляет заказами, а веб-интерфейс на порту 5002 выступает в роли шлюза, через который пользователь взаимодействует с системой.

В User Service реализованы стандартные R/W/D операции для пользователей, но с одной важной особенностью: при запросе данных конкретного

пользователя сервис сам обращается к Order Service через HTTP, чтобы подтянуть его заказы. Order Service тоже имеет базовые эндпоинты для работы с заказами, а данные хранятся в словаре, где заказы группируются по ID пользователя.

Для удобства взаимодействия был создан веб-интерфейс с HTML-страницей, на которой расположены формы для добавления пользователей и заказов, а также таблицы для отображения существующих записей с кнопками удаления. Этот сервис принимает запросы от пользователя и перенаправляет их к соответствующим микросервисам.

После завершения реализации было проведено тестирование системы. Был создан новый пользователь через POST запрос, затем для него добавлен заказ, проверено получение информации о пользователе вместе с его заказами, и наконец выполнено удаление пользователя с проверкой, что его заказы тоже исчезли. Все запросы выполнялись как через командную строку с помощью curl, так и через разработанный веб-интерфейс.

Пример работы с ИС:

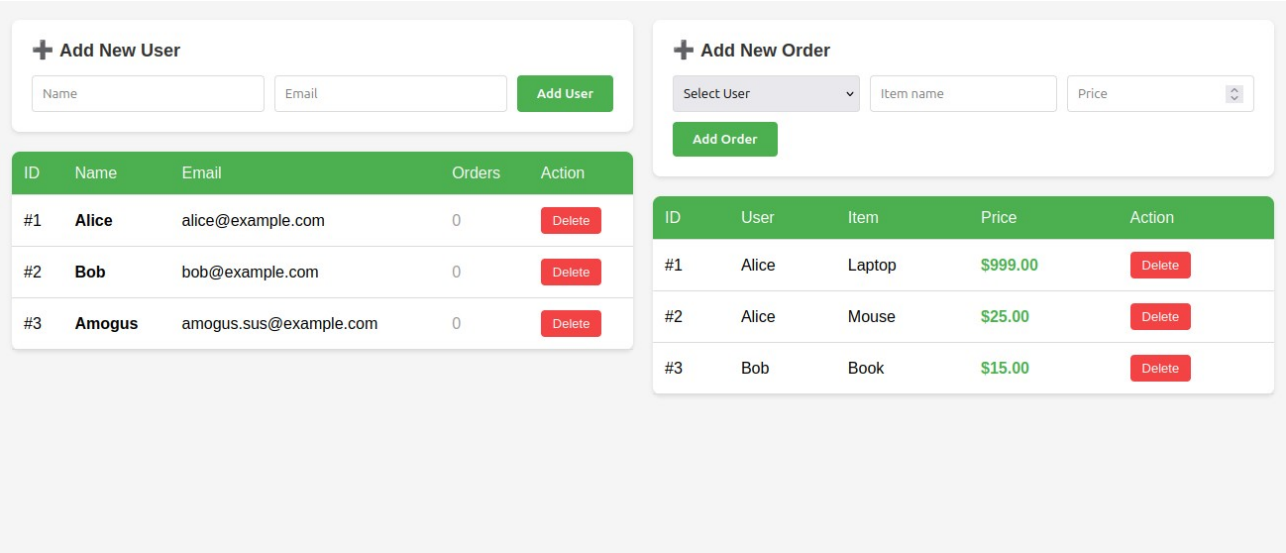

рис.1 Графический веб-интерфейс

```
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
curl -i -X GET http://localhost:5000/users
HTTP/1.1 200 OK
Server: Werkzeug/3.0.1 Python/3.12.3
Date: Tue, 03 Mar 2026 09:34:08 GMT
Content-Type: application/json
Content-Length: 209
Connection: close

{
  "1": {
    "email": "alice@example.com",
    "name": "Alice"
  },
  "2": {
    "email": "bob@example.com",
    "name": "Bob"
  },
  "3": {
    "email": "amogus.sus@example.com",
    "name": "Amogus"
  }
}
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
```

рис.2 Получение списка пользователей

```
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
curl -i -X GET http://localhost:5000/users/1
HTTP/1.1 200 OK
Server: Werkzeug/3.0.1 Python/3.12.3
Date: Tue, 03 Mar 2026 09:35:43 GMT
Content-Type: application/json
Content-Length: 212
Connection: close

{
  "email": "alice@example.com",
  "name": "Alice",
  "orders": [
    {
      "id": 1,
      "item": "Laptop",
      "price": 999
    },
    {
      "id": 2,
      "item": "Mouse",
      "price": 25
    }
  ]
}
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
```

рис.3 Информация об одном из пользователей включая заказы

```
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
curl -i -X POST -H 'Content-Type: application/json' -d '{"
name": "Charlie", "email": "charlie@example.com"}' http://
localhost:5000/users
HTTP/1.1 201 CREATED
Server: Werkzeug/3.0.1 Python/3.12.3
Date: Tue, 03 Mar 2026 09:37:15 GMT
Content-Type: application/json
Content-Length: 91
Connection: close

{
  "id": 100,
  "user": {
    "email": "charlie@example.com",
    "name": "Charlie"
  }
}
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
```

рис.4 Добавление пользователя

```
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
curl -i -X POST -H 'Content-Type: application/json' -d '{"
item": "Keyboard", "price": 75}' http://localhost:5001/ord
ers/1
HTTP/1.1 201 CREATED
Server: Werkzeug/3.0.1 Python/3.12.3
Date: Tue, 03 Mar 2026 09:38:49 GMT
Content-Type: application/json
Content-Length: 53
Connection: close

{
  "id": 100,
  "item": "Keyboard",
  "price": 75
}
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
```

рис.5 Добавление заказа

```
reference_cat@referenceCatPC:~/Projects/RS_Labs/Lab2/src$
curl -i -X GET http://localhost:5001/orders
HTTP/1.1 200 OK
Server: Werkzeug/3.0.1 Python/3.12.3
Date: Tue, 03 Mar 2026 09:39:38 GMT
Content-Type: application/json
Content-Length: 311
Connection: close

{
  "1": [
    {
      "id": 1,
      "item": "Laptop",
      "price": 999
    },
    {
      "id": 2,
      "item": "Mouse",
      "price": 25
    },
    {
      "id": 100,
      "item": "Keyboard",
      "price": 75
    }
  ],
  "2": [
    {
      "id": 3,
      "item": "Book",
      "price": 15
    }
  ]
}
```

рис.5 Просмотр всех заказов

Вывод:

В ходе выполнения лабораторной работы было разработано распределенное приложение, состоящее из двух независимых микросервисов и веб-интерфейса. User Service отвечает за управление пользователями, Order Service обрабатывает заказы, а веб-интерфейс выступает в роли шлюза для взаимодействия с пользователем. Каждый микросервис имеет собственную базу данных и предоставляет REST API для доступа к своей функциональности, что полностью соответствует требованиям микросервисной архитектуры.

Ключевым результатом работы стала успешная реализация сетевого взаимодействия между сервисами: User Service при получении информации о пользователе отправляет запрос к Order Service для получения связанных заказов. Также была продемонстрирована отказоустойчивость системы — при недоступности одного из сервисов остальные продолжают корректно функционировать. Разработанный веб-интерфейс позволяет наглядно продемонстрировать все базовые сценарии совместной работы сервисов: создание пользователей и заказов, просмотр связанных данных и удаление записей.

Таким образом, поставленные задачи выполнены в полном объеме — создано работающее распределенное приложение на микросервисной архитектуре, соответствующее всем функциональным требованиям. Полученные навыки проектирования и реализации микросервисов, организации их взаимодействия через сеть и обеспечения независимости компонентов могут быть применены при разработке более сложных распределенных систем

Приложение. Код программы

app.py

```
from flask import Flask, render_template, request, redirect, url_for
import requests
import json

app = Flask(__name__)

USER_SERVICE = 'http://localhost:5000'
ORDER_SERVICE = 'http://localhost:5001'

@app.route('/')
def index():
    # Get all users
    try:
        users_response = requests.get(f'{USER_SERVICE}/users', timeout=3)
        users = users_response.json() if users_response.status_code == 200 else {}
    except:
        users = {}

    # Get all orders
    try:
        orders_response = requests.get(f'{ORDER_SERVICE}/orders', timeout=3)
        orders = orders_response.json() if orders_response.status_code == 200 else {}
    except:
        orders = {}

    return render_template('index.html', users=users, orders=orders)
```

```
@app.route('/add_user', methods=['POST'])
def add_user():
    name = request.form.get('name')
    email = request.form.get('email')

    if name and email:
        try:
            requests.post(f'{USER_SERVICE}/users',
                          json={'name': name, 'email': email},
                          timeout=3)
        except:
            pass

    return redirect(url_for('index'))

@app.route('/add_order', methods=['POST'])
def add_order():
    user_id = request.form.get('user_id')
    item = request.form.get('item')
    price = request.form.get('price')

    if user_id and item and price:
        try:
            requests.post(f'{ORDER_SERVICE}/orders/{user_id}',
                          json={'item': item, 'price': float(price)},
                          timeout=3)
        except:
            pass
```

```
    return redirect(url_for('index'))

@app.route('/delete_user/<int:user_id>')
def delete_user(user_id):
    try:
        requests.delete(f'{USER_SERVICE}/users/{user_id}', timeout=3)
    except:
        pass
    return redirect(url_for('index'))

@app.route('/delete_order/<int:order_id>')
def delete_order(order_id):
    try:
        requests.delete(f'{ORDER_SERVICE}/orders/order/{order_id}', timeout=3)
    except:
        pass
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(port=5002, debug=True)
```

order_service.py

```
from flask import Flask, jsonify, request
import requests

app = Flask(__name__)

orders = {
    1: [{"id": 1, "item": "Laptop", "price": 999},
```

```
        {"id": 2, "item": "Mouse", "price": 25}],
    2: [{"id": 3, "item": "Book", "price": 15}]
}

next_order_id = 100


@app.route('/orders/<int:user_id>', methods=['GET'])
def get_user_orders(user_id):
    user_orders = orders.get(user_id, [])
    return jsonify(user_orders)


@app.route('/orders', methods=['GET'])
def get_all_orders():
    return jsonify(orders)


@app.route('/orders/<int:user_id>', methods=['POST'])
def create_order(user_id):
    global next_order_id
    data = request.get_json()

    if not data or 'item' not in data or 'price' not in data:
        return jsonify({"error": "Item and price required"}), 400

    if user_id not in orders:
        orders[user_id] = []

    new_order = {
```

```
        "id": next_order_id,
        "item": data['item'],
        "price": data['price']
    }
    orders[user_id].append(new_order)
    next_order_id += 1

    return jsonify(new_order), 201


@app.route('/orders/order/<int:order_id>', methods=['DELETE'])
def delete_order(order_id):
    # Delete order by ID across all users
    for user_id, user_orders in orders.items():
        for i, order in enumerate(user_orders):
            if order['id'] == order_id:
                deleted = user_orders.pop(i)
                return jsonify({"message": "Order deleted", "order": deleted})
    return jsonify({"error": "Order not found"}), 404


@app.route('/orders/user/<int:user_id>', methods=['DELETE'])
def delete_user_orders(user_id):
    if user_id in orders:
        del orders[user_id]
        return jsonify({"message": f"All orders for user {user_id} deleted"})
    return jsonify({"error": "User not found"}), 404


if __name__ == '__main__':
```

```
    app.run(port=5001, debug=True)
```

user_service.py

```
from flask import Flask, request, jsonify
import requests
import time

app = Flask(__name__)

users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"},
    3: {"name": "Amogus", "email": "amogus.sus@example.com"},
}

next_user_id = 100

@app.route('/users', methods=['GET'])
def get_users():
    return jsonify(users)

@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = users.get(user_id)

    if not user:
        return jsonify({"Error": "User not found"}), 404

    # fetching orders
```

```
    try:
        orders_response = requests.get(f'http://localhost:5001/orders/{user_id}')
        if orders_response.status_code == 200:
            user['orders'] = orders_response.json()
    except:
        return jsonify({"Error": "Fetching orders failed!"}), 404

    return jsonify(user)


@app.route('/users', methods=['POST'])
def create_user():
    global next_user_id
    data = request.get_json()

    if not data or 'name' not in data or 'email' not in data:
        return jsonify({"error": "Name and email required"}), 400

    user_id = next_user_id
    users[user_id] = {
        "name": data['name'],
        "email": data['email']
    }
    next_user_id += 1

    return jsonify({"id": user_id, "user": users[user_id]}), 201


@app.route('/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
```

```
    if user_id in users:
        del users[user_id]
        # Optionally notify order service to delete user's orders
        try:
            requests.delete(f'http://localhost:5001/orders/user/{user_id}', timeout=2)
        except:
            pass  # Order service might be down, that's ok
        return jsonify({"message": "User deleted"})
    return jsonify({"error": "User not found"}), 404


if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

README.md

```
# Running
install `flask` and `requests` via pip

Terminal A: `python3 user_service.py`\
Terminal B: `python3 order_service.py`\
Terminal C: `python3 app.py`

# Users service API calls examples
#### Get all users:
`curl -i -X GET http://localhost:5000/users`

#### Get specific user with orders (user service calls order service):
`curl -i -X GET http://localhost:5000/users/1` where `1` is an user id
```

Delete user:

`curl -i -X DELETE http://localhost:5000/users/3` where `3` is an user id

Add user:

`curl -i -X POST -H 'Content-Type: application/json' -d '{"name": "Charlie", "email": "charlie@example.com"}' http://localhost:5000/users`

Orders service API calls examples

Get all orders

`curl -i -X GET http://localhost:5001/orders`

Get all orders from user:

`curl -i -X GET http://localhost:5001/orders/1` where `1` is an user id

Delete order:

`curl -i -X DELETE http://localhost:5001/orders/order/3` where `3` is an order id

Delete order for specific user:

`curl -i -X DELETE http://localhost:5001/orders/user/3` where `3` is an user id

Add order:

`curl -i -X POST -H 'Content-Type: application/json' -d '{"item": "Keyboard", "price": 75}' http://localhost:5001/orders/1` where `1` is an user id